Solving the Traveling Salesman Problem with Simulated Annealing and Genetic Algorithms

CSCI 580 — Assignment 4 (SA + GA) · Technical Report

Name: _____ · October 2026

Abstract

A 40-city Euclidean TSP instance is solved with two metaheuristics: Simulated Annealing (SA) using a random 2-opt neighborhood and geometric cooling, and a Genetic Algorithm (GA) on permutation tours using tournament selection, Order Crossover (OX), swap mutation and elitism. Starting from a nearest-neighbor (NN) tour of length 6.0214, SA reached **5.0057 (−16.87%)** with 20,001 tour evaluations and the GA reached **5.3672 (−10.86%)** with 80,200 evaluations; both pass the required 10% improvement threshold. Instrumentation shows that SA escapes 2-opt local optima through accepted uphill moves, while the GA loses almost all population diversity within 25 generations. As extra work, replacing swap mutation with inversion mutation lets the GA match SA (mean 4.926 vs 4.916).

1. Problem Context

The Traveling Salesman Problem asks for the shortest closed tour that visits each of n cities exactly once. It is NP-hard: the number of distinct tours is $(n-1)!/2$, about 10^{46} for $n = 40$, so exhaustive search is impossible and heuristic optimization is used instead. The instance here is generated reproducibly by `make_cities(n=40, seed=7)`: 40 points uniformly distributed in the unit square, with Euclidean distances. A solution is a permutation of city indices, and its cost is the closed-tour length.

Two baselines/representations are shared by both methods: the greedy nearest-neighbor heuristic (always walk to the closest unvisited city), which gives a quick but suboptimal tour of length 6.0214 with visible edge crossings (Fig. 1), and the 2-opt move, which reverses a segment of the tour. Reversing a segment replaces exactly two edges, and any tour with crossing edges can be shortened by some 2-opt move.

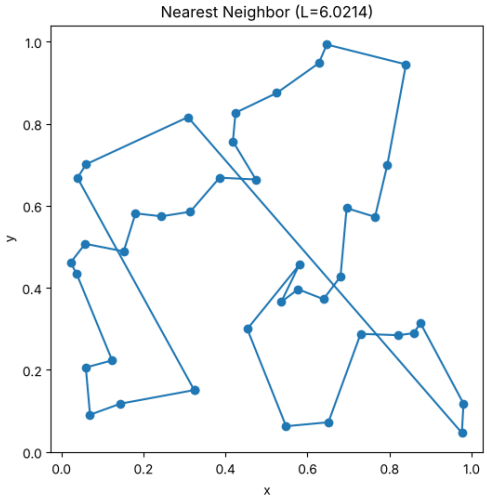


Figure 1. Nearest-neighbor baseline tour (L = 6.0214). Note the long closing edges and crossings.

2. Solution Methods

2.1 Simulated Annealing

SA performs a random walk over tours. At each step it proposes a random 2-opt neighbor and computes ΔE = L(candidate) − L(current). Improving moves are always accepted; worsening moves are accepted with the Metropolis probability exp(−ΔE/T). The temperature T starts at t0 and is multiplied by α every iteration (geometric cooling), clamped at 10^{-12} to avoid division by zero. At high T the walk accepts many uphill moves (exploration); as T → 0 it becomes greedy (exploitation). The completed TODOs:

```
        accept = False
        # Acceptance rule (Metropolis criterion)
        if delta <= 0:
            accept = True
        else:
            accept = rng.random() < math.exp(-delta / T)
        ...
        # Cooling schedule: geometric cooling, clamped
        T = max(T * cfg.alpha, 1e-12)
```

The function also takes an optional `stats` dictionary that records the ΔE of every accepted and rejected move, the number of accepted uphill moves, and the current length and temperature at every iteration (used in Section 4).

2.2 Genetic Algorithm

The GA evolves a population of 200 permutation tours (the NN tour plus 199 random tours). Each generation copies the 5 best tours unchanged (elitism) and fills the rest by selecting two parents with tournament selection, applying OX crossover with probability 0.9 and swap mutation with probability 0.2.

```
def tournament_select(pop, lengths, k, rng):
    # sample k distinct individuals, return a copy of the shortest
    contenders = rng.sample(range(len(pop)), k)
    winner = min(contenders, key=lambda i: lengths[i])
    return pop[winner][:]

def order_crossover_ox(parent1, parent2, rng):
    n = len(parent1)
    a, b = sorted(rng.sample(range(n), 2))
    child = [None] * n
    child[a:b + 1] = parent1[a:b + 1]           # 1) keep a slice of parent 1
    used = set(parent1[a:b + 1])
    fill = [c for c in parent2[b + 1:] + parent2[:b + 1] if c not in used]
    positions = list(range(b + 1, n)) + list(range(0, a))
    for pos, c in zip(positions, fill):         # 2) fill the rest in parent-2 order,
        child[pos] = c                          #    starting after the slice (wrapping)
    return child

def mutate_swap(tour, rng):
    t = tour[:]
    i, j = rng.sample(range(len(t)), 2)
    t[i], t[j] = t[j], t[i]
    return t
```

OX keeps a contiguous block of parent 1 and the *relative order* of the remaining cities from parent 2, so the child is always a valid permutation. The GA function was extended with an optional `stats` argument (population average/worst/std of lengths, number of distinct tours, mean pairwise edge distance, cumulative evaluations) and optional `crossover_fn`/`mutate_fn`

arguments that default to OX and swap and are used only for the Part 7 experiments. Unit checks assert that every operator returns a valid permutation.

2.3 Diversity metric

Two GA diversity measures are tracked. **Distinct tours** counts unique tours after normalizing rotation and direction. **Edge distance** between two tours is the fraction of edges they do *not* share (0 = identical, ~1 = unrelated random tours); the mean over 300 random pairs is reported every generation.

3. Experimental Setup and Main Results

Method	Parameters	Seed
SA	iters = 20,000, t0 = 0.2, α = 0.9995 (final T ≈ 9.1e-6), init = NN tour	123
GA	pop = 200, generations = 400, tournament k = 5, crossover = 0.9, mutation = 0.2, elite = 5, init = NN + 199 random	999

Method	Best length	Improvement	Evaluations	Runtime
Nearest neighbor	6.0214	—	1	—
Simulated Annealing	**5.0057**	**16.87%**	**20,001**	**0.18 s**
Genetic Algorithm	5.3672	10.86%	80,200	3.43 s

Both methods pass the notebook's `grade_check` (≥ 10% improvement over NN). Figures 2–4 show the final tours and the provided history plots.

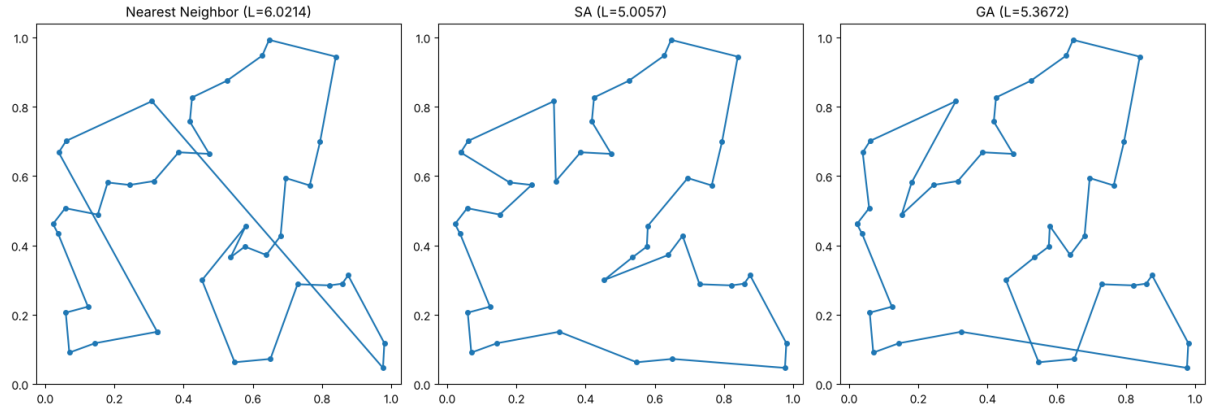


Figure 2. Final tours: NN (6.0214), SA (5.0057), GA (5.3672). SA's tour has no crossings; the GA tour still contains detours inherited from the NN seed.

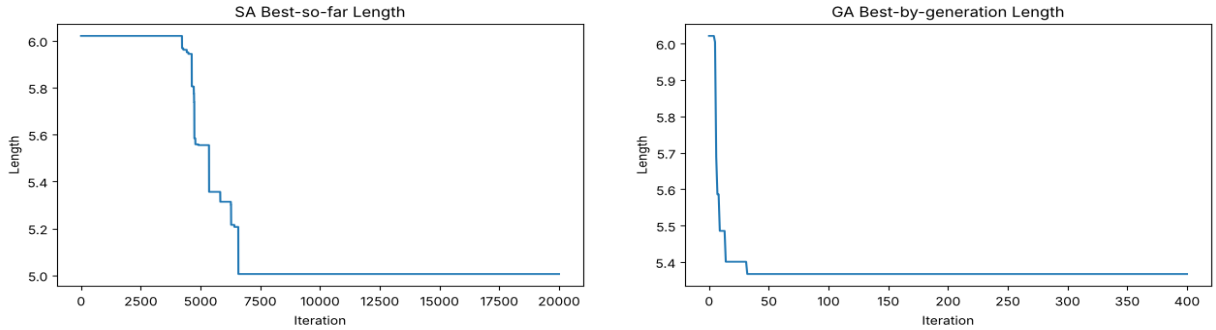


Figure 3. Best-so-far length — SA per iteration (left) and GA per generation (right).

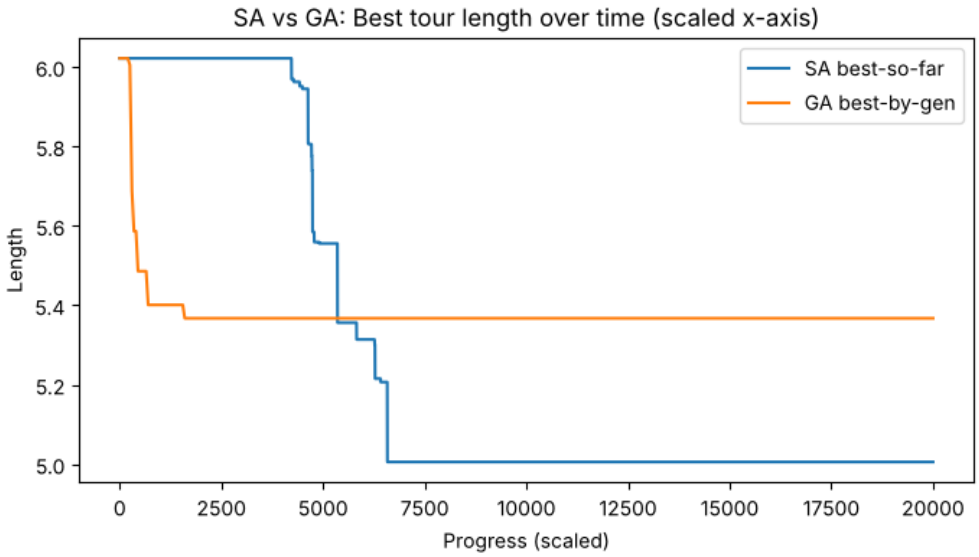


Figure 4. The provided plot_compare overlay (x-axis scaled so both runs span the same width).

4. Data Collection and Analysis (Part 6)

4.1 SA: ΔE of accepted vs rejected moves

Of 20,000 proposed moves, 608 (3.0%) were accepted: 324 improving, 1 neutral and **283 uphill**. Most proposals are rejected because a random 2-opt move on a good tour usually adds long edges. Accepted uphill moves are concentrated at small ΔE, which is exactly the Metropolis shape: exp(−ΔE/T) quickly becomes negligible for large ΔE, so large uphill moves are almost always rejected (Fig. 5).

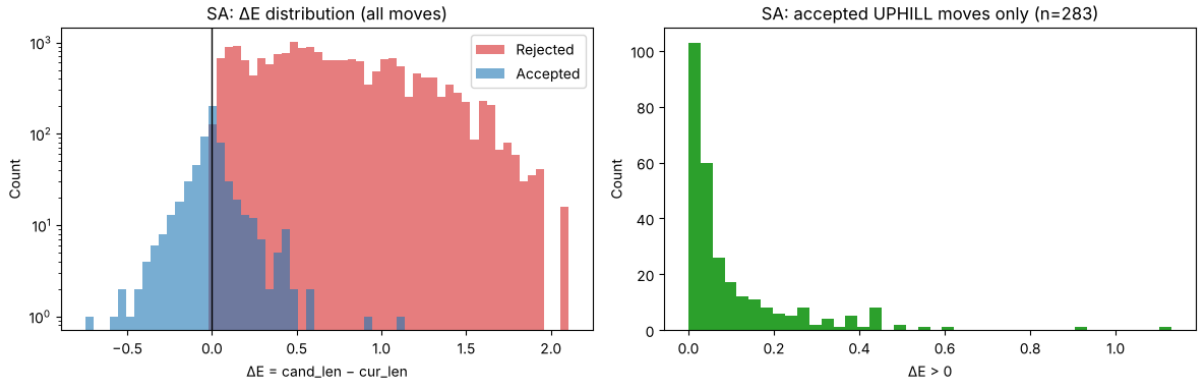


Figure 5. Left: ΔE histogram of all accepted vs rejected moves (log count). Right: only accepted uphill moves.

4.2 SA: current vs best length

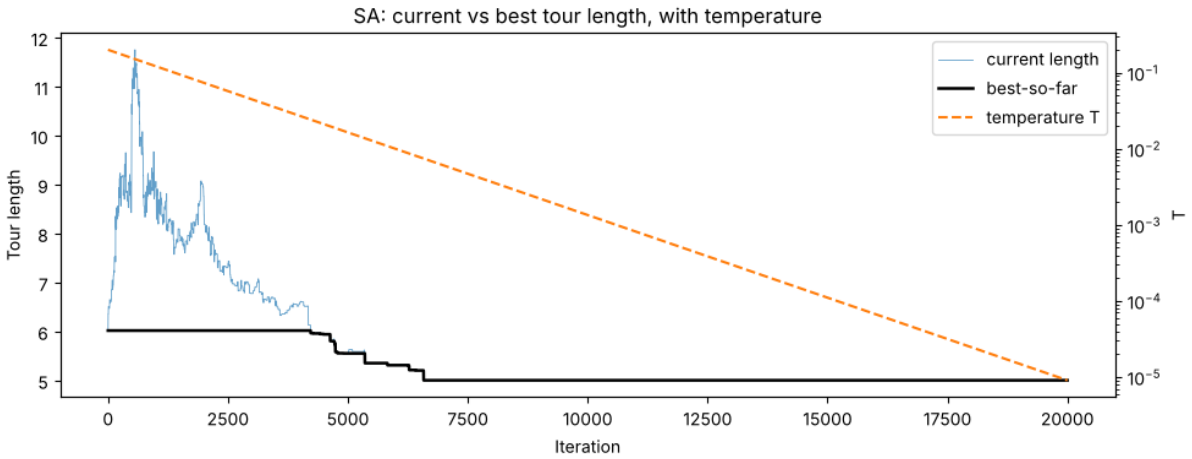


Figure 6. SA current length (blue), best-so-far (black) and temperature (orange, log axis).

At t0 = 0.2 the walk initially *leaves* the NN tour, rising to a current length of 11.7 by iteration ~600. As T falls below ~0.03 (iteration ~4,000) it descends, passes the starting length, and sets 20 new bests between iterations ~4,200 and 6,623. After T ≈ 0.003 no uphill moves are accepted and the run is frozen at 5.0057.

4.3 GA: population fitness and diversity

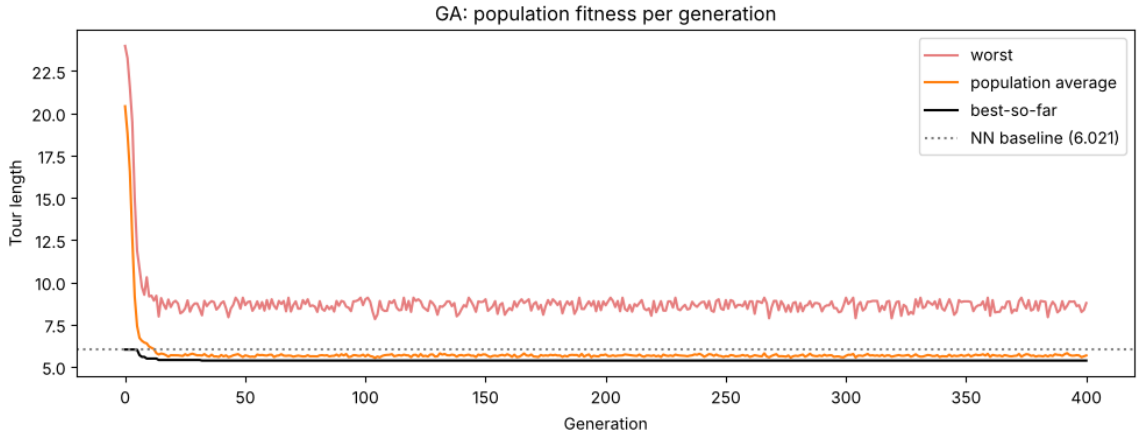


Figure 7. GA worst, population-average and best length per generation.

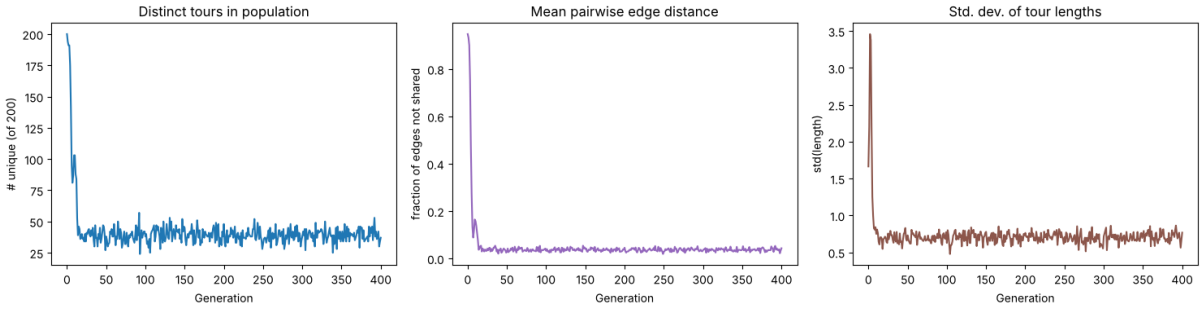


Figure 8. GA diversity: distinct tours, mean pairwise edge distance, std. dev. of lengths.

Generation	0	25	50	100	200	400
Distinct tours (of 200)	200	41	38	38	48	37
Mean edge distance	0.947	0.042	0.029	0.033	0.046	0.043

Population average	20.416	5.743	5.673	5.611	5.728	5.669
Best	6.021	5.401	5.367	5.367	5.367	5.367

4.4 Normalized by evaluations, and variability over seeds

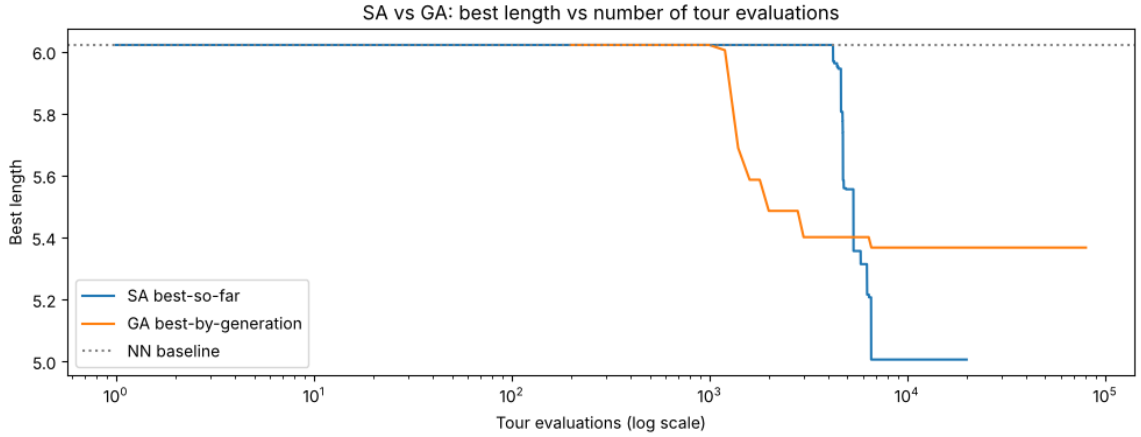


Figure 9. Best length vs number of tour evaluations (log x-axis). One GA generation = 200 evaluations.

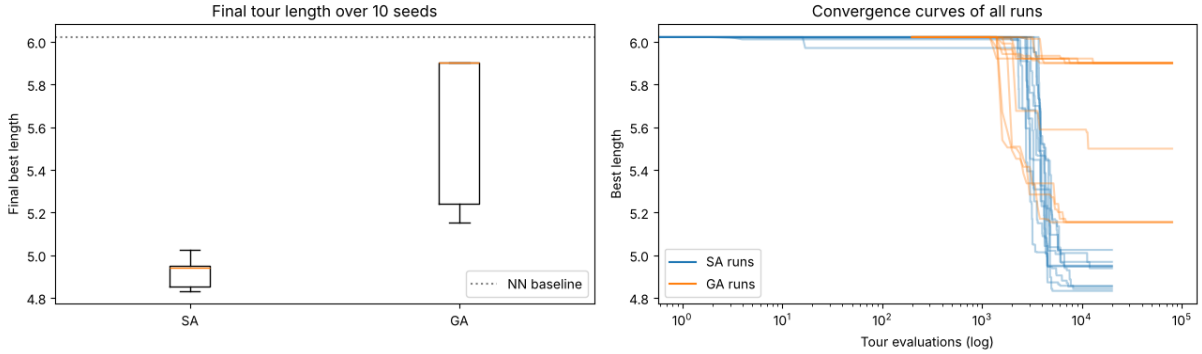


Figure 10. Ten independent seeds per method: final lengths (left) and all convergence curves (right).

10 seeds	Mean	Std	Min	Max
SA	4.9162	0.0618	4.8326	5.0248
GA	5.6363	0.3365	5.1546	5.9001

4.5 Answers to the Part 6 questions

Which algorithm converged faster? The GA *stopped improving* sooner: it came within 1% of its own final value after ~3,000 evaluations (15 generations) and its best was flat from generation ~50 to 400. But this is premature convergence to a worse tour. Per evaluation, SA reached good tours faster: it beat the GA's final length (5.367) after only ~5,355 evaluations — about 1/15 of the GA's budget — and was within 1% of its own, better final value after ~6,580 evaluations. SA was also ~19× faster in wall-clock time.

Which showed more variability? Across seeds, the GA: its final-length standard deviation (0.337) is more than 5× SA's (0.062), with results ranging from 5.15 to 5.90; some GA runs barely leave the NN seed's basin. Within one run, SA is the noisier process — its current length swings up to 11.7 at high temperature — while the GA's best is a smooth staircase.

Did SA escape local minima? Evidence. Yes. (1) 283 uphill moves were accepted. (2) Fig. 6 shows the walk leaving the NN tour and returning to a much lower region, with all new bests found after the walk had been above the best-so-far. (3) Control experiment: the identical 2-opt search at T = 0 (greedy hill climbing, which stops at the first 2-opt local optimum) averaged **5.293** over 10 seeds versus **4.957** for SA from the same start. SA reliably ends below the local optima that trap greedy search.

Did GA lose diversity? Evidence. Yes, early and almost completely. Distinct tours fell from 200 to 41 by generation 25, and mean edge distance from 0.947 to 0.042, meaning two random individuals share ~96% of their edges (Fig. 8). The population average sits just above the best for 350 generations without the best improving. The NN seed is ~3× shorter than a random tour (6.0 vs 20.4), so under elitism and k = 5 tournaments its descendants take over within a few generations; afterwards OX between near-identical parents reproduces the parents and swap mutation, which breaks up to four edges, almost always makes a tour worse.

Which worked better and why? SA — shorter tours, 5× less run-to-run variance, ¼ of the evaluations and ~1/19 of the runtime. Its 2-opt move matches the problem structure (it changes exactly two edges and removes crossings directly) and the cooling schedule controls the exploration–exploitation trade-off explicitly. The required GA's operators are not adjacency-aware, so once diversity collapses its search effectively stops.

5. Optional Work (Part 7)

5.1 Alternative GA operators

Three alternatives were implemented: **inversion mutation** (reverse a random segment, i.e. a 2-opt move), **PMX crossover** (Partially Mapped Crossover), and a **memetic GA** that polishes the best elite with greedy 2-opt every generation. Each variant was run with 5 seeds at the same 200 × 400 budget; two variants were also run without the NN seed.

```
def mutate_inversion(tour, rng):
    return random_two_opt_neighbor(tour, rng)   # reverse a random segment
```

GA variant (5 seeds)	Mean	Std	Best
OX + swap (required)	5.647	0.316	5.168
PMX + swap	5.900	0.000	5.900
OX + inversion	**4.926**	**0.079**	**4.833**
PMX + inversion	5.129	0.091	5.042
OX + swap, no NN seed	5.759	0.379	5.200
OX + inversion, no NN seed	5.060	0.138	4.833
Memetic (OX + inversion + 2-opt)	5.305	0.142	5.131
SA reference (10 seeds)	4.916	0.062	4.833

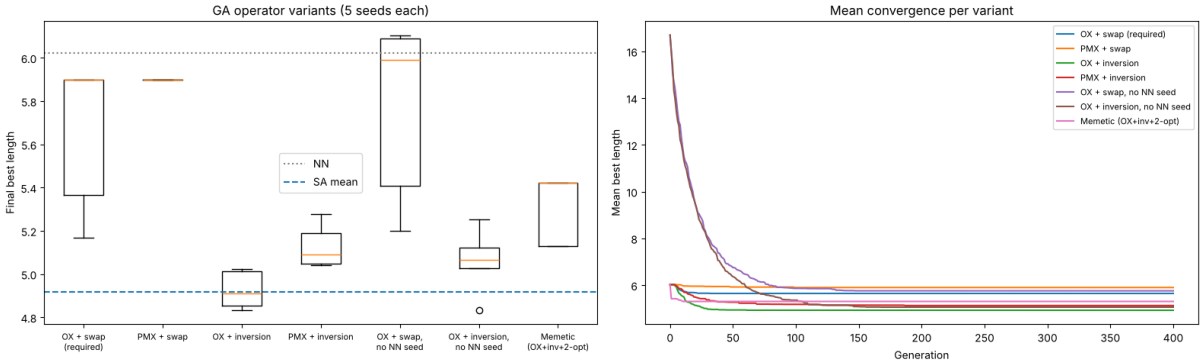


Figure 11. GA operator variants: final lengths (left) and mean convergence (right).

- **Mutation matters most.** Replacing swap with inversion improved the mean by 0.72 (≈13%) and cut the spread 4×; OX + inversion matches SA's mean and finds the same best tour (4.8326). Inversion changes only two edges, so it explores without destroying good adjacencies.
- **OX beats PMX.** Tour length depends on adjacency, not absolute position. OX preserves relative order; PMX preserves positions. PMX + swap converged to the identical tour (5.9001) in all five seeds.
- **The NN seed helps on average but accelerates convergence.** Without it, OX + inversion still averages 5.06 and once reaches 4.8326, so the GA is doing genuine optimization work.
- **Memetic underperformed** (5.305). Polishing the elite gives it a large fitness lead, increasing selection pressure and collapsing diversity within ~10 generations. Local search needs diversity protection (smaller tournaments, occasional polishing) to pay off.

5.2 SA parameter sensitivity

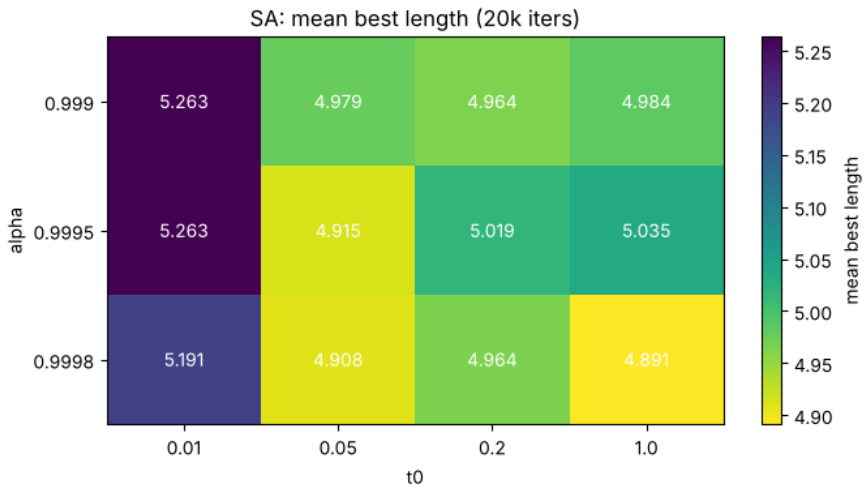


Figure 12. Mean SA best length (3 seeds) for t0 × α, 20,000 iterations each.

With t0 = 0.01 SA is nearly greedy (5.19–5.26). For every t0 ≥ 0.05 the mean lies in 4.89–5.04, so SA is robust to its parameters once the starting temperature allows some uphill moves. The best setting (t0 = 1.0, α = 0.9998, final T ≈ 0.018) spends more iterations in the productive medium-temperature range.

6. Conclusions

- Both methods improve substantially on the NN baseline: SA by 16.9% and the required GA by 10.9%.
- SA was the better method for this instance: better tours, much lower variance, fewer evaluations, and far lower runtime. Its accepted uphill moves and the T = 0 control experiment show it escapes 2-opt local optima.
- The required GA suffers premature convergence: diversity collapses within ~25 generations, driven by the strong NN seed and a destructive swap mutation.
- Operator choice is decisive for GAs on TSP: an adjacency-preserving inversion mutation lets the GA match SA. Operators that respect problem structure matter more than added machinery such as local search.

Reproducibility

All results come from `TSP_SA_and_GA_Assignment.ipynb` in the Assignment_4 folder of the GitHub repository; running all cells top-to-bottom (≈75 s) reproduces every number and figure, since every run uses a fixed random seed.